



Universidad autónoma de baja california

Ingeniería en computación

Internet de las cosas

Taller-lab 5: obtener hora mediante internet

Aguilar Noriega Leocundo

Garcia Chaves erik

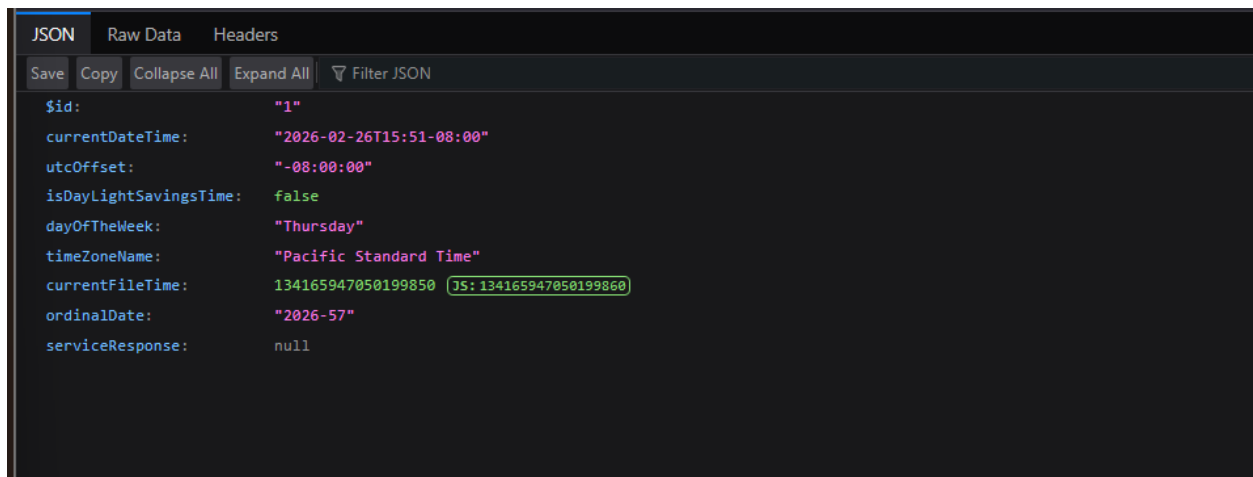
Viernes 27 de febrero del 2026

Parte 1: obtención de hora actual mediante HTTP.

para la realización de la parte 1, fue necesario encontrar un sitio el cual este sobre HTTP para que la petición sea de una manera mas sencilla que por HTTPS.

El servicio que utilizaremos es: <http://worldclockapi.com/api/json/pst/now>

con el cual obtendremos una respuesta en formato JSON:



```
JSON | Raw Data | Headers
Save Copy Collapse All Expand All Filter JSON
{
  $id: "1",
  currentDateTime: "2026-02-26T15:51-08:00",
  utcOffset: "-08:00:00",
  isDayLightSavingsTime: false,
  dayOfTheWeek: "Thursday",
  timeZoneName: "Pacific Standard Time",
  currentFileTime: 134165947050199850 JS: 134165947050199860,
  ordinalDate: "2026-57",
  serviceResponse: null
}
```

Utilizando la configuración de HTTP cliente, realizamos una petición GET a la API, en la cual vamos a esperar una respuesta, donde si es correcta el código de status será 200 en caso contrario nos dará el código de error.

Por lo que tomamos la respuesta y lo regresamos a la función principal con el propósito de parsear la respuesta con solo el trozo que nos interesa el cual es la hora actual.

```

har *get_time(void) {

    esp_http_client_config_t config = {
        .url = "http://worldclockapi.com/api/json/pst/now",
        .method = HTTP_METHOD_GET,
    };

    esp_http_client_handle_t cliente = esp_http_client_init(&config);

    // Abre la conexión sin consumir el body
    esp_err_t err = esp_http_client_open(cliente, 0);
    char *buffer = NULL;
    if(err == ESP_OK){
        int content_length = esp_http_client_fetch_headers(cliente);
        int status = esp_http_client_get_status_code(cliente);

        ESP_LOGI(TAG, "HTTP status: %d, tamaño: %d", status, content_length);

        if(content_length > 0){
            buffer = (char*)malloc(content_length + 1);
            int leidos = esp_http_client_read(cliente, buffer, content_length);
            buffer[leidos] = '\0';
            ESP_LOGI(TAG, "Respuesta: %s", buffer);
        }
    } else {
        ESP_LOGE(TAG, "Error al abrir conexión: %s", esp_err_to_name(err));
    }

    esp_http_client_close(cliente);
    esp_http_client_cleanup(cliente);

    return buffer;
}

```

La segunda función que utilizamos es la que parsea la respuesta buscando la hora y tan solo se muestra por terminal

```

MAIN: {"$id":"1","currentDateTime":"2026-02-26T18:34-08:00","utcOffset":"-08:00:00","isDayLightSavingsTime":false,"dayOfTheWeek":"Thursday","timeZoneName":"Pacific Standard Time","currentFileTime":134166044935365083,"ordinalDate":"2026-57","serviceResponse":null}
I (8284) get time / tijuana: parse time ---> 18:34
I (8334) wifi:<ba-add>idx:0 (ifx:0, d8:6d:17:90:8f:48), tid:0, ssn:4, winSize:64

```

generalidades del protocolo HTTP en ESP32

el protocolo HTTP es el protocolo por el cual nos permite realizar peticiones de datos y recursos en la web. se compone de una estructura cliente-servidor por lo que el inicio de la petición es iniciada por el que recibe los datos, el cliente. Los mensajes enviados por el cliente se les llama *peticiones* y las respuestas del servidor *respuestas*. Este es un protocolo en la *capa de aplicación* se transmite sobre el protocolo *TCP*.

Parte 2: petición API hora mediante TCP/sockets

Como el servicio esta sobre HTTP, HTTP esta sobre TCP por lo que se puede hacer la petición directamente mediante sockets, esto puede tener mayor control sobre el flujo, la conexión, HTTP nos da la trama ya armada y los sockets TCP uno es quien la construye.

Por lo que es una gran ventaja para los que desean realizar algo muy específico.

Lo que primero es resolver el **host** para obtener la dirección IP del servicio.

Después viene el flujo de la conexión mediante sockets

Tenemos la función **socket** que en donde si se crea correctamente nos devuelve el descriptor del socket un numero entero positivo, en otro caso -1.

La estructura **sockaddr_in** indicamos la dirección destino

La función **connect** establece la conexión TCP.

Después mandamos la petición la cual es un método **GET**

Con **recv** recibimos la respuesta del servidor.

Cunado realizamos la petición mediante **HTTP** la API nos devolverá muchas cosas, tanto el JSON como el header, por lo que necesitamos eliminar ese header para quedarnos con el JSON donde estará la hora en esa respuesta.

La función para encontrar y parsear la respuesta es básicamente la misma.

```

char *get_time_tcp(void){

    //resolvemos el DNS
    struct hostent *server = gethostbyname(HOST);
    if (server == NULL) {
        ESP_LOGE(TAG, "DNS fallo");
        return NULL;
    }

    //2 creamos el socket
    /**
     * primer parametro indica la familia de direccion, AF_INET para IPv4 y AF_INET6 para IPv6
     *
     * el segundo parametro indica que tipo de socket sera, entonces este es un <
     * por lo que establece una conexi3n de extremo a extremo, este envia datos si
     * y recibe los datos en el orden de envio.
     *
     *
     * tercer parametro indica el protocolo, por lo general se pone 0 para que el
     * predeterminado segun la familia.
     *
     * @return -> retorna el descriptor del socket, entero no negativo
     * @return -> -1 en caso de error
     */
    int sock = socket(AF_INET, SOCK_STREAM, 0);

    if(sock < 0){
        ESP_LOGE(TAG, "ERROR al crear el socket");
        return NULL;
    }

    //paso 3; configurar direccion y conectar
    struct sockaddr_in dest;
    dest.sin_family = AF_INET;
    dest.sin_port=htons(SERVER_PORT);
    memcpy(&dest.sin_addr.s_addr, server->h_addr, server->h_length);

    if (connect(sock, (struct sockaddr *)&dest, sizeof(dest)) != 0) {
        ESP_LOGE(TAG, "error a conectarlo");
        close(sock);
        return NULL;
    }

    ESP_LOGI(TAG, "conectando a %s", HOST);

    //enviar la peticion HTTP cruda

    send(sock, REQUEST,strlen(REQUEST), 0);

    //recibir los datos

    char response[1024]={0};
    int total =0, bytes=0;
    do {
        bytes = recv(sock, response + total, sizeof(response) - total - 1, 0);
        if (bytes > 0) total += bytes;
    } while (bytes > 0);
    response[total] = '\0';

    close(sock); // 6. Cerrar socket

    // Separar header del body buscando \r\n\r\n
    char *body = strstr(response, "\r\n\r\n");
    if (body == NULL) {
        ESP_LOGE(TAG, "No se encontro el body");
        return NULL;
    }
}

```

Tendremos la siguiente respuesta:

```
I (27584) TCP: time--> {"$id":"1","currentDateTime":"2026-02-26T18:05-08:00","utcOffset":"-08:00:00","isDayLightSavingsTime":false,"dayOfTheWeek":"Thursday","timeZoneName":"Pacific Standard Time","currentFileTime":134166027131991324,"ordinalDate":"2026-57","serviceResponse":null}
I (27684) TCP: conectando a worldclockapi.com
```

Este es el JSON que nos devuelve en donde debemos parsear, lo que nos interesa es la segunda propiedad en donde viene la fecha y hora.

```
I (7544) TCP: time--> {"$id":"1","currentDateTime":"2026-02-26T18:13-08:00","utcOffset":"-08:00:00","isDayLightSavingsTime":false,"dayOfTheWeek":"Thursday","timeZoneName":"Pacific Standard Time","currentFileTime":134166027131991324,"ordinalDate":"2026-57","serviceResponse":null}
I (7564) TCP:
CURRENT TIME---> 18:13
I (7834) wifi:<ba-add>idx:0 (ifx:0, d8:6d:17:90)
```

Nos da la hora actual



Generalidades de TCP

Es un protocolo para establecer comunicación y envío de paquetes a través de internet y garantizar la entrega exitosa de datos y mensajes a través de la red.

Es uno de los protocolos más comunes utilizados dentro de las comunicaciones de red digitales y garantiza la entrega de datos de extremo a extremo.

Este protocolo establece una conexión, un puente activo y abierto constantemente hasta que se finalice en envío de paquetes

Conclusiones

Durante la practica la practica se pudo poner a prueba 2 comunicaciones en entornos web muy utilizados, durante las pruebas pude notar que la comunicación directa mediante sockets TCP es mucho más rápido y establece una conexión casi sin fallas a diferencia de HTTP en donde muchas veces no puedo establecer la conexión y nos indica fallas constantes en la conexión. Es un poco más fácil de programar con HTTP, pero mayor seguridad y eficiencia TCP.

Bibliografía:

- <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
- <https://www.ibm.com/docs/es/i/7.6.0?topic=characteristics-socket-type>
- https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_client.html